开放-课件5 3D Gaussian Splatting算法安装与实测

论文：3D Gaussian Splatting for Real-Time Radiance Field Rendering

项目地址：https://github.com/graphdeco-inria/gaussian-splatting

目录导航

cvlife.net内部资料

1 Overview and Requirements

如果在Windows用，有视频逐步骤指导，参考youtube视频链接

本文档后续内容都是建立在Ubuntu上进行使用的

下面内容都摘自README.md，纯英文，我们在第二节将展示实测安装算法遇到的问题和全流程

1.1 Codebase

The codebase has 4 main components:

- A **PyTorch-based optimizer** to produce a 3D Gaussian model from SfM inputs
- A **network viewer** that allows to connect to and visualize the optimization process
- An **OpenGL-based** real-time viewer to render trained models in real-time.
- A **script** to help you turn your own images into optimization-ready SfM data sets

The components have different requirements w.r.t. both hardware and software. They have been tested on Windows 10 and Ubuntu Linux 22.04. Instructions for setting up and running each of them are found in the sections below.

> 作者只在Ubuntu 22.04做了测试；实际我们在Ubuntu 18.04和20.04进行了测试
>
> 在Ubuntu18.04可以完成训练，但无法可视化；Ubuntu20.04可以可视化

1.2 Hardware Requirements

- CUDA-ready GPU with Compute Capability 7.0+
- 24 GB VRAM (to train to paper evaluation quality)
- Please see FAQ for smaller VRAM configurations

> 有24G显存最好，11G显存在小范围少图场景7K次迭代也可以完成训练，大范围多图场景不行

1.3 Software Requirements

- Conda (recommended for easy setup)
- C++ Compiler for PyTorch extensions (we used Visual Studio 2019 for Windows)
- CUDA SDK 11 for PyTorch extensions, install *after* Visual Studio (**we used 11.8, known issues with 11.6**)
- C++ Compiler and CUDA SDK must be compatible

> CUDA 11.8很重要

关于Conda环境：

Conda环境（相比Miniconda，更推荐Anaconda）安装参考链接：

【精选】Ubuntu:conda的安装与使用_乌班图使用conda-CSDN博客

Anaconda | The World’s Most Popular Data Science Platform

cvlife.net内部资料

2 实际安装测试

2.1 拉取代码

使用--recursive把submodules给全部拉取下来，这个很重要；不加的话，后续有的东西装不上

```
# SSH
git clone git@github.com:graphdeco-inria/gaussian-splatting.git --recursive
```

or

```
# HTTPS
git clone https://github.com/graphdeco-inria/gaussian-splatting --recursive
```

如果submodule安装失败，可能出现以下报错

```
/usr/include/c++/11/bits/std_function.h:435:145: error: parameter packs not expanded with ‘…’:
435 | function(_Functor&& __f) | ^
/usr/include/c++/11/bits/std_function.h:435:145: note: ‘_ArgTypes’
/usr/include/c++/11/bits/std_function.h:530:146: error: parameter packs not expanded with ‘…’: 530 | operator=(_Functor&& __f) | ^
/usr/include/c++/11/bits/std_function.h:530:146: note: ‘_ArgTypes’
```

可以手动安装子模块

```
pip install submodules/diff-gaussian-rasterization

pip install submodules/simple-knn
```

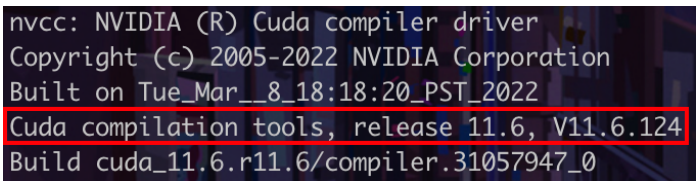
cvlife.net内部资料

2.2 CUDA环境对安装的影响

首先确定你的cuda版本：

nvcc -V 或 cat /usr/local/cuda/version.txt

如果看不到cuda版本，你需要安装适配你显卡型号的cuda



2.2.1 CUDA 11.6试验

我们一开始是用cuda11.6做的测试

如果你能够正常执行以下命令不报错，这是最好的事情

```
conda env create --file environment.yml
conda activate gaussian_splatting
```

如果你遇到如下图这样的报错或其他报错，是非常正常的

```
  error: subprocess-exited-with-error

  × python setup.py egg_info did not run successfully.
  │ exit code: 1
  ╰─> [12 lines of output]
      Traceback (most recent call last):
        File "<string>", line 36, in <module>
        File "<pip-setuptools-caller>", line 34, in <module>
        File "/home/wangshouan/Nerf_Based_Slam_Code/gaussian-splatting/submodules/diff-gaussian-rasterization/setup.py", line 13, in <module>
          from torch.utils.cpp_extension import CUDAExtension, BuildExtension
        File "/home/wangshouan/anaconda3/envs/gaussian_splatting/lib/python3.7/site-packages/torch/__init__.py", line 201, in <module>
          _load_global_deps()
        File "/home/wangshouan/anaconda3/envs/gaussian_splatting/lib/python3.7/site-packages/torch/__init__.py", line 154, in _load_global_deps
          ctypes.CDLL(lib_path, mode=ctypes.RTLD_GLOBAL)
        File "/home/wangshouan/anaconda3/envs/gaussian_splatting/lib/python3.7/ctypes/__init__.py", line 364, in __init__
          self._handle = _dlopen(self._name, mode)
      OSError: /home/wangshouan/anaconda3/envs/gaussian_splatting/lib/python3.7/site-packages/torch/lib/../../../../libcublas.so.11: symbol cublasLtGetStatusString
      [end of output]

  note: This error originates from a subprocess, and is likely not a problem with pip.
error: metadata-generation-failed

× Encountered error while generating package metadata.
╰─> See above for output.

note: This is an issue with the package mentioned above, not pip.
hint: See above for details.

failed

CondaEnvException: Pip failed

wangshouan@ict-Precision-7920-Tower:~/Nerf_Based_Slam_Code/gaussian-splatting$ conda env create --file environment.yml

CondaValueError: prefix already exists: /home/wangshouan/anaconda3/envs/gaussian_splatting

wangshouan@ict-Precision-7920-Tower:~/Nerf_Based_Slam_Code/gaussian-splatting$ conda activate gaussian_splatting
```

环境创建了，但是没有安装完毕

经过查看issue和readme，我们可以得到结论：**cuda 11.8非常重要！**

需要**非常注意！！！！3dgs对CUDA版本有特殊的要求**，安装到本地的完整版CUDA版本是11.8，而安装到conda环境中供pytorch使用的阉割版本是11.6（这个版本不包含nvcc，一些复杂功能也没有），官方的解释如下：

> Hi,
>
> there’s two different things. One is the full CUDA SDK, including the compiler (NVCC). We need it to build the PyTorch extensions that we wrote ourselves and that the optimizer uses. The other is the CUDA runtime that PyTorch is built against (the latest CUDA runtime that PyTorch 1.12 works with is 11.6). If you install PyTorch with CUDA with Conda, it installs a small subset of the full CUDA SDK that cannot do compilation. If you install the full 11.6 SDK and try to compile our extensions with it, it can fail because of a known issue with C++14 support in the 11.6 CUDA compilers.

于是我们新安装了cuda 11.8

2.2.2 CUDA 11.8试验

换成cuda 11.8就**一切顺遂**

```
File  Edit  View  Search  Terminal  Help
Processing ./submodules/diff-gaussian-rasterization
  Preparing metadata (setup.py): started
  Preparing metadata (setup.py): finished with status 'done'
Processing ./submodules/simple-knn
  Preparing metadata (setup.py): started
  Preparing metadata (setup.py): finished with status 'done'
Building wheels for collected packages: diff-gaussian-rasterization, simple-knn
  Building wheel for diff-gaussian-rasterization (setup.py): started
  Building wheel for diff-gaussian-rasterization (setup.py): still running...
  Building wheel for diff-gaussian-rasterization (setup.py): finished with status 'done'
  Created wheel for diff-gaussian-rasterization: filename=diff_gaussian_rasterization-0.0.0-cp37-cp37m-linux_x86_64.whl size
=e6ebb82dfaf7824c12aa1f87cb961616c1a5ab4e4348fab63321a4c5c26d3718
  Stored in directory: /home/wangshouan/.cache/pip/wheels/12/a6/c9/d5d2b5c1786999337eb1a119caec95ea5ad4132d27514af9da
  Building wheel for simple-knn (setup.py): started
  Building wheel for simple-knn (setup.py): still running...
  Building wheel for simple-knn (setup.py): finished with status 'done'
  Created wheel for simple-knn: filename=simple_knn-0.0.0-cp37-cp37m-linux_x86_64.whl size=1746564 sha256=90f702a0e38b3d02e
d7327f64bf3eaf5c7d42a43dfb33d17
  Stored in directory: /home/wangshouan/.cache/pip/wheels/88/2b/76/bffee8b633158cb1cb44109c116cae8b051ba3901750f4abe2
Successfully built diff-gaussian-rasterization simple-knn
Installing collected packages: simple-knn, diff-gaussian-rasterization
Successfully installed diff-gaussian-rasterization-0.0.0 simple-knn-0.0.0

done
#
# To activate this environment, use
#
#     $ conda activate gaussian_splatting
#
# To deactivate an active environment, use
#
#     $ conda deactivate

wangshouan@ict-Precision-7920-Tower:~/Nerf_Based_Slam_Code/gaussian-splatting$
```

非常顺利的执行完毕

单独安装pytorch的命令

```
pip install torch==1.13.0+cu118 -f https://download.pytorch.org/whl/torch_stable.html
```

如果你的系统默认CUDA无法更改成11.8或者不方便更改，也是有办法使用CUDA11.8的

详细流程参见 https://www.cnblogs.com/smileglaze/p/16826946.html **，在bashrc里调整当前用户的cuda**

cvlife.net内部资料

3 使用自己的数据调用算法

3.1 colmap安装

参照这里https://blog.csdn.net/gootyking/article/details/134533251

安装colmap，先安装依赖库

```
Selecting previously unselected package libklu1:amd64.
Preparing to unpack .../15-libklu1_1%3a5.1.2-2_amd64.deb ...
Unpacking libklu1:amd64 (1:5.1.2-2) ...
Selecting previously unselected package libldl2:amd64.
Preparing to unpack .../16-libldl2_1%3a5.1.2-2_amd64.deb ...
Unpacking libldl2:amd64 (1:5.1.2-2) ...
Selecting previously unselected package libumfpack5:amd64.
Preparing to unpack .../17-libumfpack5_1%3a5.1.2-2_amd64.deb ...
Unpacking libumfpack5:amd64 (1:5.1.2-2) ...
Selecting previously unselected package librbio2:amd64.
Preparing to unpack .../18-librbio2_1%3a5.1.2-2_amd64.deb ...
Unpacking librbio2:amd64 (1:5.1.2-2) ...
Selecting previously unselected package libsuitesparse-dev:amd64.
Preparing to unpack .../19-libsuitesparse-dev_1%3a5.1.2-2_amd64.deb ...
Unpacking libsuitesparse-dev:amd64 (1:5.1.2-2) ...
Selecting previously unselected package libceres-dev.
Preparing to unpack .../20-libceres-dev_1.13.0+dfsg0-2_amd64.deb ...
Unpacking libceres-dev (1.13.0+dfsg0-2) ...
Selecting previously unselected package libcgal13:amd64.
Preparing to unpack .../21-libcgal13_4.11-2build1_amd64.deb ...
Unpacking libcgal13:amd64 (4.11-2build1) ...
Selecting previously unselected package libgmpxx4ldbl:amd64.
Preparing to unpack .../22-libgmpxx4ldbl_2%3a6.1.2+dfsg-2ubuntu0.1_amd64.deb ...
Unpacking libgmpxx4ldbl:amd64 (2:6.1.2+dfsg-2ubuntu0.1) ...
Selecting previously unselected package libgmp-dev:amd64.
Preparing to unpack .../23-libgmp-dev_2%3a6.1.2+dfsg-2ubuntu0.1_amd64.deb ...
Unpacking libgmp-dev:amd64 (2:6.1.2+dfsg-2ubuntu0.1) ...
Selecting previously unselected package libmpfr-dev:amd64.
Preparing to unpack .../24-libmpfr-dev_4.0.1-1_amd64.deb ...
Unpacking libmpfr-dev:amd64 (4.0.1-1) ...
Selecting previously unselected package libcgal-dev:amd64.
Preparing to unpack .../25-libcgal-dev_4.11-2build1_amd64.deb ...
Unpacking libcgal-dev:amd64 (4.11-2build1) ...
```

github官方的colmap库是3.8版本的。2023年1月放出的

编译这个版本遇到了freeimage等报错信息

如果你也遇到了这个问题

你可以**退回到3.7版本**，在github库的release里找到

还是一样的编译流程，这次就再没有问题

Colmap的sudo make install成功

```
-- Installing: /usr/local/include/colmap/lib/SiftGPU/SiftPyramid.h
-- Installing: /usr/local/include/colmap/lib/SiftGPU/ProgramCU.h
-- Installing: /usr/local/include/colmap/lib/SiftGPU/ProgramCL.h
-- Installing: /usr/local/include/colmap/lib/SiftGPU/GLTexImage.h
-- Installing: /usr/local/include/colmap/lib/SiftGPU/ProgramCG.h
-- Installing: /usr/local/include/colmap/lib/SiftGPU/CuTexImage.h
-- Installing: /usr/local/include/colmap/lib/SiftGPU/SiftMatch.h
-- Installing: /usr/local/include/colmap/lib/SiftGPU/PyramidCL.h
-- Installing: /usr/local/include/colmap/lib/SiftGPU/ShaderMan.h
-- Installing: /usr/local/include/colmap/lib/SiftGPU/GlobalUtil.h
-- Installing: /usr/local/include/colmap/lib/SiftGPU/CLTexImage.h
-- Installing: /usr/local/include/colmap/lib/SiftGPU/PyramidGL.h
-- Installing: /usr/local/include/colmap/lib/SiftGPU/ProgramGPU.h
-- Installing: /usr/local/include/colmap/lib/SiftGPU/PyramidCU.h
-- Installing: /usr/local/share/colmap/COLMAPConfig.cmake
-- Installing: /usr/local/share/colmap/COLMAPConfigVersion.cmake
-- Installing: /usr/local/share/colmap/cmake
-- Installing: /usr/local/share/colmap/cmake/FindFreeImage.cmake
-- Installing: /usr/local/share/colmap/cmake/FindGlog.cmake
-- Installing: /usr/local/share/colmap/cmake/FindEigen3.cmake
-- Installing: /usr/local/share/colmap/cmake/FindGlew.cmake
-- Installing: /usr/local/share/applications/COLMAP.desktop
-- Installing: /usr/local/lib/colmap/libflann.a
-- Installing: /usr/local/lib/colmap/libgraclus.a
-- Installing: /usr/local/lib/colmap/liblsd.a
-- Installing: /usr/local/lib/colmap/libpba.a
-- Installing: /usr/local/lib/colmap/libpoisson_recon.a
-- Installing: /usr/local/lib/colmap/libsift_gpu.a
-- Installing: /usr/local/lib/colmap/libsqlite3.a
-- Installing: /usr/local/lib/colmap/libvlfeat.a
-- Installing: /usr/local/lib/colmap/libcolmap.a
-- Installing: /usr/local/lib/colmap/libcolmap_cuda.a
-- Installing: /usr/local/bin/colmap
wangshouan@ict-Precision-7920-Tower:~/Nerf_Based_Slam_Code/colmap-3.7/build$
```

cylife.net内部资料

3.2 拍视频还是拍照

从效果好的角度，**尽量得到这个场景中所有位置所有角度的图像**

这个准备过程的仔细程度对最终模型的效果起到重要作用

为了满足上述要求，还是**拍视频最好，完了再抽帧**

训练的照片要分辨率**统一**，相机参数、光照等参数都要统一

分辨率推荐720p、1080p

3.3 自己录制视频并抽帧

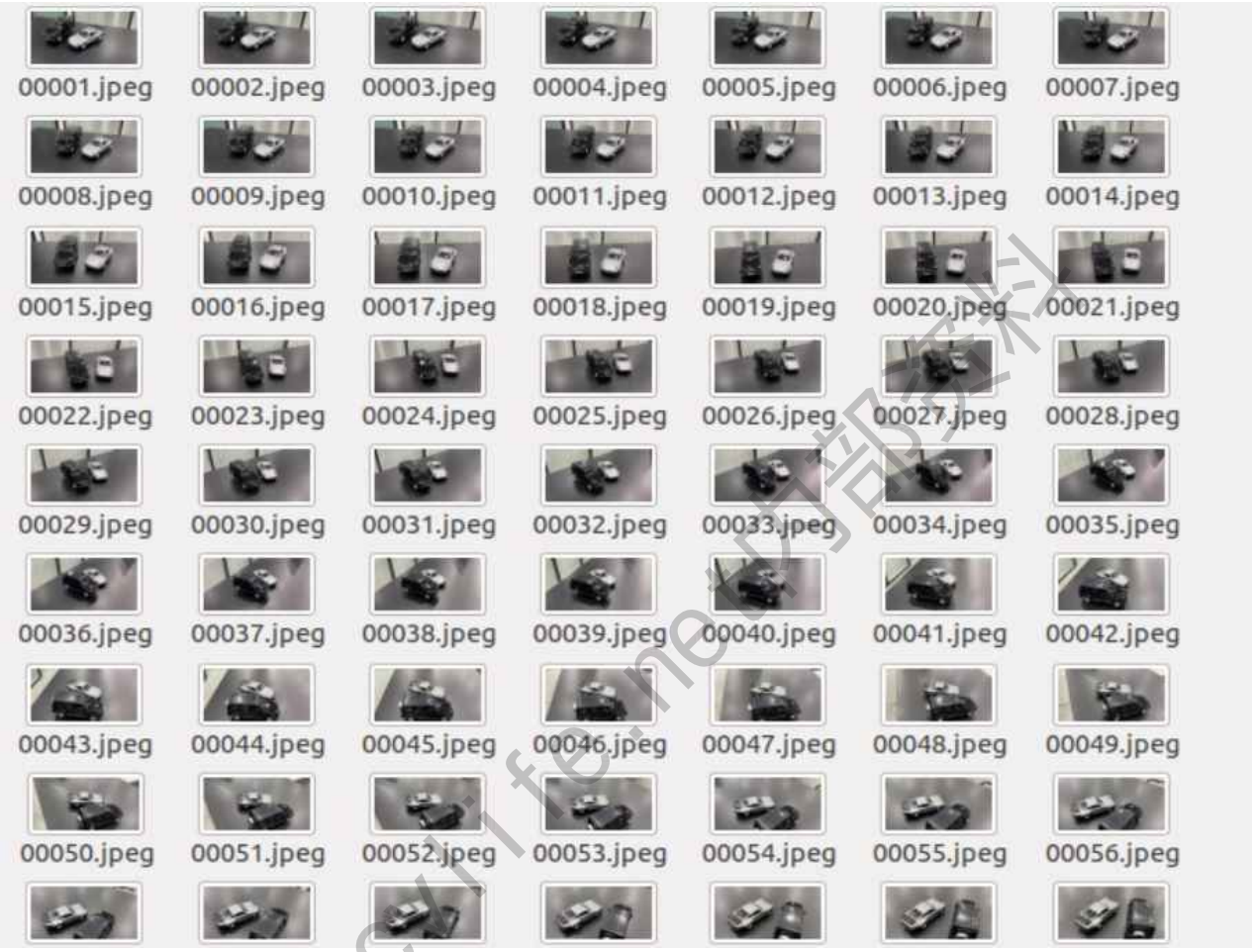
先建一个data目录，data目录里需要建个input文件夹，名字必须是这个input

xxxx.mp4是你录制的视频

视频抽帧命令如下

```
ffmpeg -i xxxx.mp4 -r 5 -q:v 2 -f image2 input/%05d.jpeg
```

> 此命令的抽帧图片质量较好
>
> -i就是输入视频
> -r就是帧率，控制抽的图片数量
> 后面input/%05d.jpg ，图片名字最终是 00001.jpg, 00002.jpg这样的格式。
>
> 最终在data/input下就有了一堆图片



cyrilfe.net内部资料

3.4 一键式生成数据

```
python convert.py -s data
```

此data就是你刚才使用到的目录，内含一个input子目录，内部是你抽帧得到的图片

下图是经典的colmap流程

```
  Camera:          #1 - OPENCV
  Focal Length:    1152.00px
  Features:        1347
Processed file [112/115]
  Name:            00112.jpeg
  Dimensions:      960 x 540
  Camera:          #1 - OPENCV
  Focal Length:    1152.00px
  Features:        1390
Processed file [113/115]
  Name:            00113.jpeg
  Dimensions:      960 x 540
  Camera:          #1 - OPENCV
  Focal Length:    1152.00px
  Features:        1595
Processed file [114/115]
  Name:            00114.jpeg
  Dimensions:      960 x 540
  Camera:          #1 - OPENCV
  Focal Length:    1152.00px
  Features:        1827
Processed file [115/115]
  Name:            00115.jpeg
  Dimensions:      960 x 540
  Camera:          #1 - OPENCV
  Focal Length:    1152.00px
  Features:        1786
Elapsed time: 0.077 [minutes]

==============================================================================
Exhaustive feature matching
==============================================================================

Matching block [1/3, 1/3]
```

```
Bundle adjustment report
------------------------
    Residuals : 6670
   Parameters : 1424
   Iterations : 3
         Time : 0.0262499 [s]
 Initial cost : 0.547985 [px]
   Final cost : 0.539577 [px]
  Termination : Convergence

  => Merged observations: 0
  => Completed observations: 2
  => Filtered observations: 2
  => Changed observations: 0.001146

==============================================================================
Registering image #106 (31)
==============================================================================

  => Image sees 315 / 613 points

Pose refinement report
----------------------
    Residuals : 614
   Parameters : 6
   Iterations : 9
         Time : 0.00824169 [s]
 Initial cost : 0.676698 [px]
   Final cost : 0.561246 [px]
  Termination : Convergence

  => Continued observations: 294
  => Added observations: 198
```

3.5 训练

```
python train.py -s data -m data/output
```

> -s还是一样，要写到data这个目录级别
> -m 是输出文件夹，就写data下的output，output目录会自动创建的

比如我这里简单的场景，7000次就完成了迭代

原始输入是iPhone11采集的视频，显卡2080Ti，115张原图送入colmap 00:12到00:29，17分钟完成训练和保存

整个训练是30000步，在7000步的时候会自动保存一个中间模型

cvlife.net内部资料

```
Undistorting image [111/115]
Undistorting image [112/115]
Undistorting image [113/115]
Undistorting image [114/115]
Undistorting image [115/115]
Writing reconstruction...
Writing configuration...
Writing scripts...
Elapsed time: 0.009 [minutes]
Done.
(gaussian_splatting) wangshouan@ict-Precision-7920-Tower:~/Nerf_Based_Slam_Code/gaussian-splatting$ python train.py -s colmap_data -m col
collections  colorama     colorsys
(gaussian_splatting) wangshouan@ict-Precision-7920-Tower:~/Nerf_Based_Slam_Code/gaussian-splatting$ python train.py -s colmap_data -m colma
p_data/output
Optimizing colmap_data/output
Output folder: colmap_data/output [16/12 00:12:03]
Tensorboard not available: not logging progress [16/12 00:12:03]
Reading camera 115/115 [16/12 00:12:03]
Converting point3d.bin to .ply, will happen only the first time you open the scene. [16/12 00:12:03]
Loading Training Cameras [16/12 00:12:03]
Loading Test Cameras [16/12 00:12:07]
Number of points at initialisation :  7945 [16/12 00:12:07]
Training progress:  23%|                                                    | 7000/30000 [03:23<12:28, 30.72it/s, Loss=0.0329419]
[ITER 7000] Evaluating train: L1 0.02588384747505188 PSNR 25.974462509155273 [16/12 00:15:30]

[ITER 7000] Saving Gaussians [16/12 00:15:30]
Training progress: 100%|                                                    | 30000/30000 [17:25<00:00, 28.71it/s, Loss=0.0230963]

[ITER 30000] Evaluating train: L1 0.01518941279500723 PSNR 29.21516265869141 [16/12 00:29:32]

[ITER 30000] Saving Gaussians [16/12 00:29:32]

Training complete. [16/12 00:29:36]
(gaussian_splatting) wangshouan@ict-Precision-7920-Tower:~/Nerf_Based_Slam_Code/gaussian-splatting$
```

> 网上找到一个例子，
>
> 分辨率 1920x1080，939张照片，convert.py的时间是二十几分钟，train.py的时间一共是15分钟。GPU显存占用22GB，训练得到的ply文件 156MB

4 交互式可视化

根据这两个issue，在ubuntu18.04上，跑3DGS算法的可视化存在一些问题

https://github.com/graphdeco-inria/gaussian-splatting/issues/314

https://github.com/graphdeco-inria/gaussian-splatting/issues/356

我实测18.04上也是运行不了

4.1 小场景的渲染效果

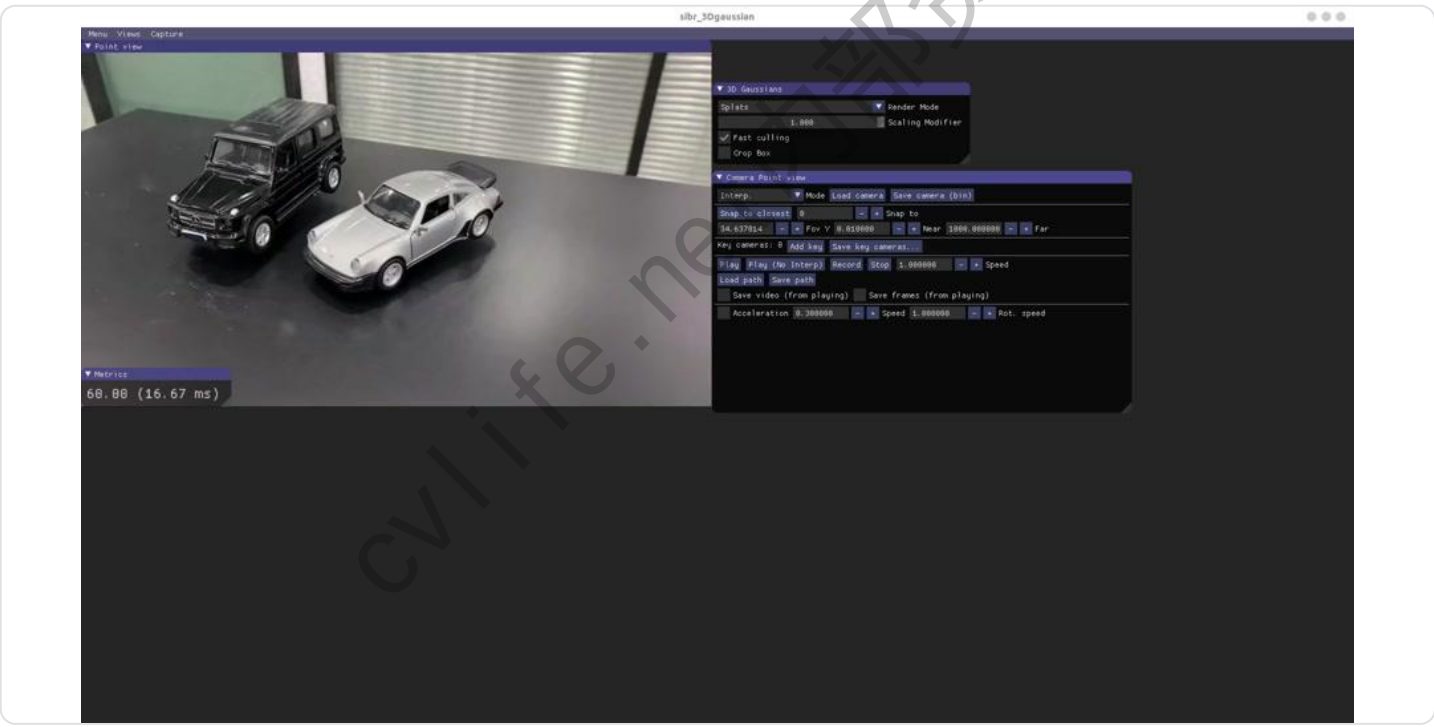
渲染是额外在Ubuntu 20.04上进行的

```
sudo apt install -y libglew-dev libassimp-dev libboost-all-dev libgtk-3-dev libopencv-dev libglfw3-dev libavdevice-dev libavcodec-dev libeigen3-dev libxxf86vm-dev libembree-dev
# Project setup
cd SIBR_viewers
git checkout fossa_compatibility #如果是22.04就不需要加这行命令
cmake -Bbuild . -DCMAKE_BUILD_TYPE=Release # add -G Ninja to build faster
cmake --build build -j24 --target install
```

这里展示自测的小场景效果

cvlife.net内部资料

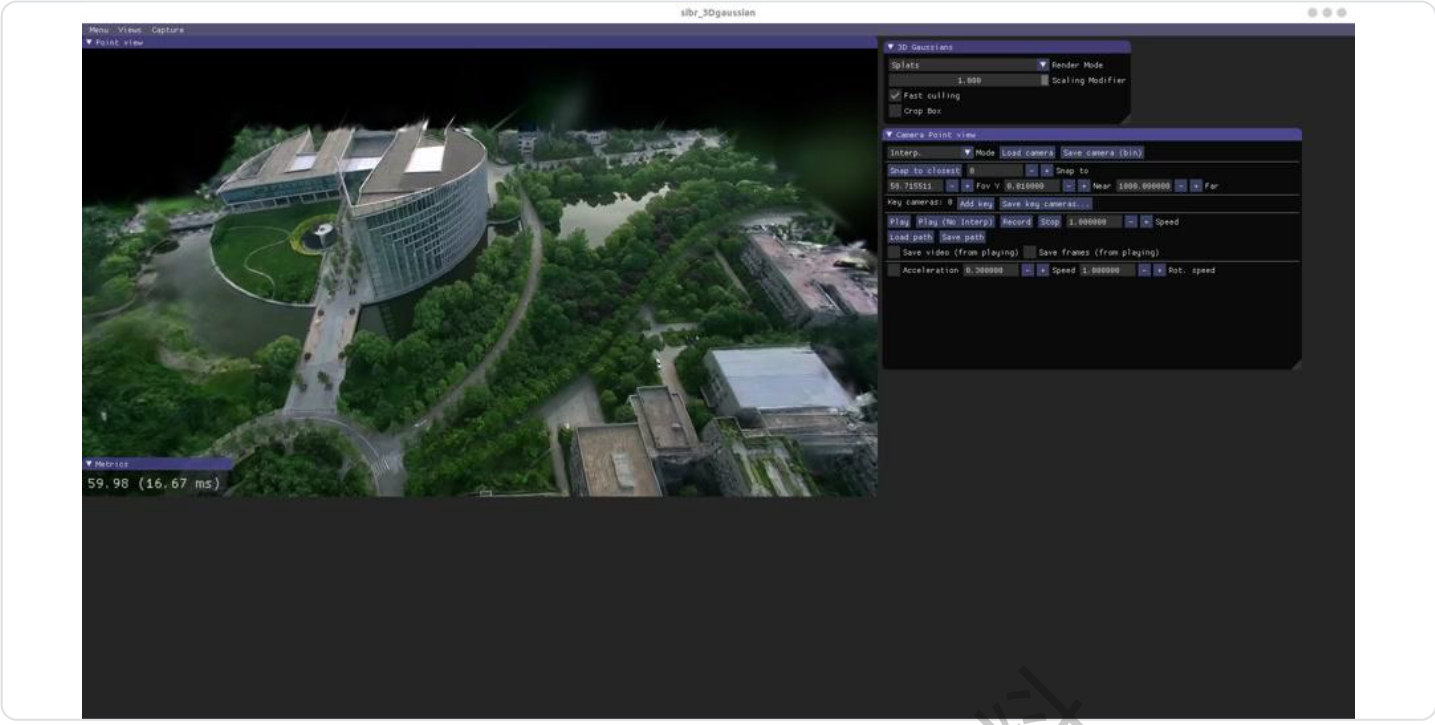
上方是纯视频，下方是Interactive Viewers整体界面



4.2 大场景的渲染效果

助教在4090显卡上运行了大场景

cvlife.net 更多资料



上方是3DGS的效果，下方是instantNGP的效果（做对比）

两种算法的输入，均为无人机航拍西北工业大学校园的视频，抽帧并SFM预处理得到图片位姿，送入4090训练